

Intermodular Projector

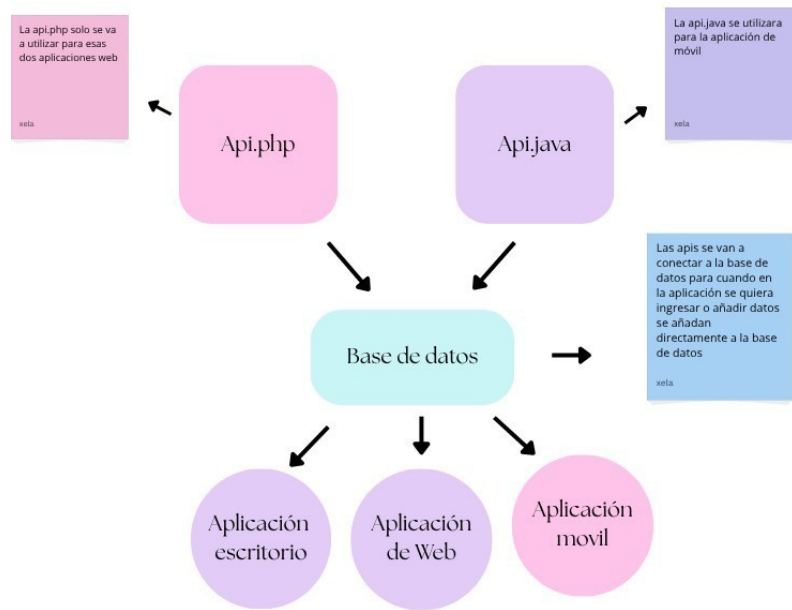
Documentation

Table of Contents:

- 1.System Architecture
- 2.API
- 3.DataBase.
- 4.Web Application.
5. Mobile Application.
6. Desktop Application.
- 7.Security
- 8.Deployment
- 9.Testing
- 10.User and Maintenance Guide
- 11.Future Improvements

1. System Architecture.

- General diagram showing how the three applications connect to the API and the database.



- Main technologies used in each part.

Aplicación de web

- PHP
- CSS
- MySQL / MariaDB
- XAMPP
- Visual Studio Code

Aplicación Movil

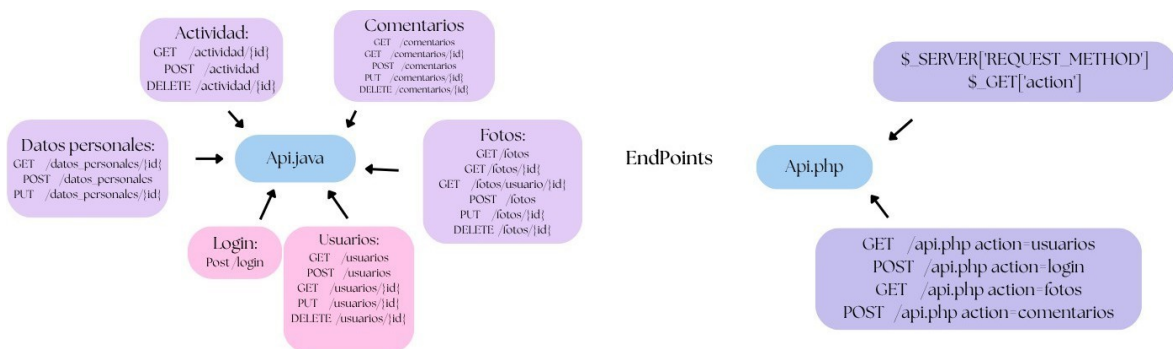
- Java
- Android Studio

Aplicación Escritorio

- C#
- PHP -> para la api
- XAMPP
- Visual Studio 2022

2.API

- List of endpoints.



HTTP methods.

- **GET:** Used to retrieve information. Not to modify data. **Example:**
`GET /users`
- **POST:** Used to create new resources.
Example: `POST /users`
- **PUT:** Used to update existing resources.
Example: `PUT /users/{id}`
- **DELETE:** Used to delete resources.
Example: `DELETE /users/{id}`

These methods allow you to implement CRUD operations (Create, Read, Update, and Delete).

- Authentication.

1. The user sends their data to the server:

`POST /login`

Content-Type: application/json

```
{  
  "user": "juan",
```

```
"password": "1234"
}
```

2. The server validates the data and returns it:

```
{
  "encrypted password": "abc123xyz"
}
```

3. The client uses this token in subsequent requests:

Authorization: Bearer abc123xyz

- Examples of requests and responses.
-

Login:

Request:

POST /login

Content-Type: application/json

```
{
  "username": "juan",
  "password": "1234"
}
```

Answer:

```
{
  "token": "abc123xyz",
  "message": "Login
successful"
}
```

Get users Request:

GET /users

Authorization: Bearer abc123xyz

Response:

```
[
{
  "id": 1,
  "name": "Juan",
  "email": "juan@email.com "
},
{
  "id": 2,
  "name": "Ana", "email":
  "ana@email.com "
}
]
```

Get user by ID Request:

GET /users/1

Authorization: Bearer abc123xyz

Response:

```
{
  "id": 1,
  "name": "Juan",
  "email": "juan@email.com "
}
```

Create user Request:

POST /users

Content-Type: application/json

```
{
  "name": "Carlos",
  "email": "carlos@email.com ",
  "password": "1234"
}
```

Response:

```
{
  "message": "User created successfully",
}
```

```
"id": 3
}
```

Update user Request:

```
PUT /users/3
Content-Type: application/json
Authorization: Bearer abc123xyz
```

```
{
  "name": "Carlos Pérez",
  "email": "carlos@email.com"
}
```

Response:

```
{
  "message": "User updated"
}
```

Delete user Request:

```
DELETE /users/3 Authorization:
Bearer abc123xyz
```

Response:

```
{
  "message": "User deleted"
}
```

3.Database

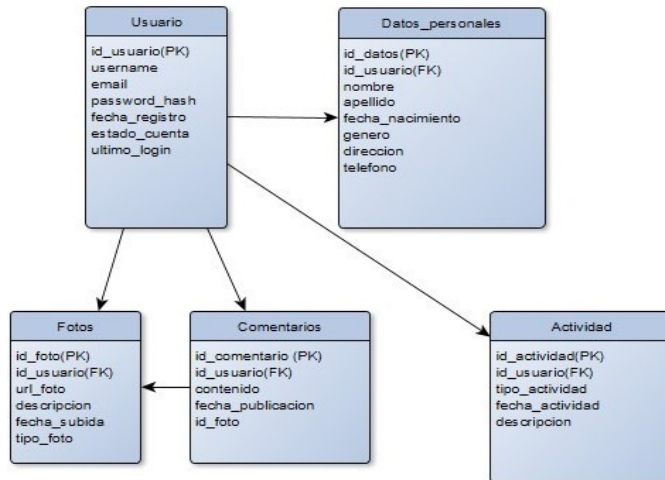
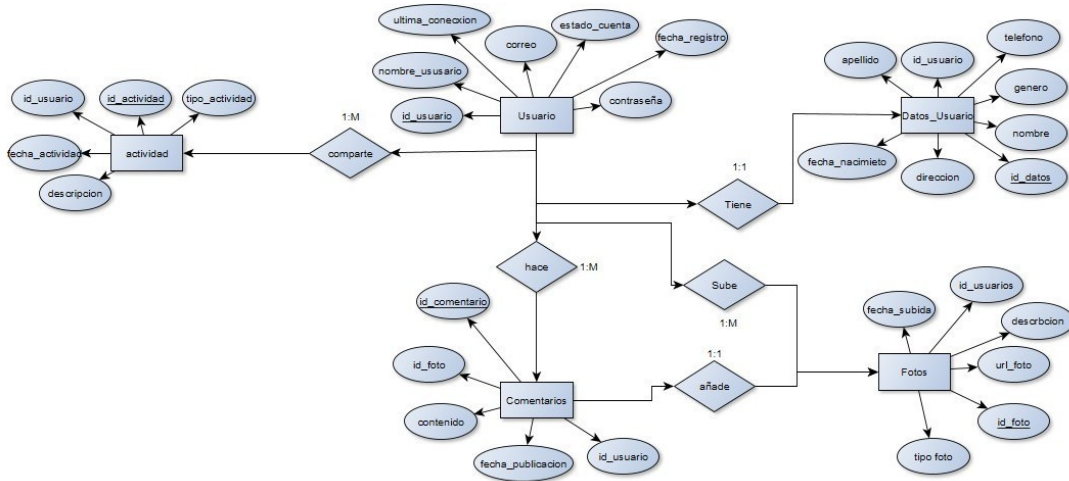


Table: User

Stores the main information about users.

- user_id (PK): Unique user identifier
 - username: User name
 - email: User's email address
 - password: Encrypted password
 - registration_date: Registration date
 - account_status: Status (active/inactive)
 - last_login: Last login
-

Table: User Data

Contains the user's personal information.

data_id (PK): Data identifier

- user_id (FK): Relationship with User
 - First name: Real first name
 - Last Name: Last Name
 - Phone: Phone
 - Gender: Gender
 - Address: Address
 - date_of_birth: Date of birth
-

Table: Photos

Stores photos uploaded by users.

- photo_id (PK): Photo identifier
- user_id (FK): User who uploaded the photo
- photo_url: Image path
- description: Photo description
- upload_date: Upload date
- photo_type: Image type

Table: Comments

Contains comments made on photos.

- comment_id (PK): Comment identifier
 - user_id (FK): User who commented
 - photo_id (FK): Photo being commented on
 - content: Comment text
 - publication_date: Date of the comment
-

Table: Activity

Records user activity.

- activity_id (PK): Identifier
- user_id (FK): User performing the activity
- activity_type: Type of action
- description: Description
- activity_date: Date

Reglas:

- Un usuario puede tener un único registro en Datos_Usuario (relación 1:1).
- Un usuario puede subir muchas fotos (relación 1).
- Una foto pertenece a un único usuario.
- Un usuario puede realizar muchos comentarios (1).
- Un comentario pertenece a un único usuario y a una única foto.
- Una foto puede tener muchos comentarios.
- Un usuario puede generar múltiples actividades.

Restricciones:

- 1.Las claves primarias (PK) son únicas y no nulas.
- 2.Las claves foráneas (FK) deben existir previamente en la tabla relacionada.
- 3.No se puede eliminar un usuario si tiene datos asociados (fotos, comentarios, etc.), salvo el borrado en cascada.
- 4.El correo debe ser único.
- 5.La contraseña debe almacenarse cifrada.

CREATE DATABASE

social_network; USE

social_network;

-- USER TABLE

CREATE TABLE User

(

user_id INT AUTO_INCREMENT PRIMARY KEY,

username VARCHAR(50) NOT NULL UNIQUE, email

VARCHAR(100) NOT NULL UNIQUE,

password_hash VARCHAR(255) NOT NULL,

registration_date DATETIME DEFAULT CURRENT_TIMESTAMP,

account_status VARCHAR(20),

last_login DATETIME

);

```

-- PERSONAL_DATA TABLE
-- 1-to-1 relationship with User CREATE
TABLE Personal_Data (
data_id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT UNIQUE,
first_name VARCHAR(50),

last_name VARCHAR(50),
date_of_birth DATE,
gender VARCHAR(20),
address VARCHAR(150),
phone_number
VARCHAR(20),
FOREIGN KEY (user_id) REFERENCES User(user_id) ON DELETE
CASCADE
);

-- PHOTOS TABLE
CREATE TABLE Photos (
photo_id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT NOT NULL,
photo_url VARCHAR(255) NOT NULL,
description VARCHAR(255),
upload_date DATETIME DEFAULT CURRENT_TIMESTAMP,
photo_type VARCHAR(50),
FOREIGN KEY (user_id) REFERENCES User(user_id) ON DELETE
CASCADE
);

-- COMMENTS TABLE
CREATE TABLE Comments (
comment_id INT AUTO_INCREMENT PRIMARY KEY, user_id INT
NOT NULL,
photo_id INT NOT NULL,
content TEXT NOT NULL,

```

```

publication_date DATETIME DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (user_id) REFERENCES User(user_id) ON DELETE
CASCADE,
FOREIGN KEY (photo_id) REFERENCES Photos(photo_id)
ON DELETE CASCADE
);

```

-- ACTIVITY TABLE

```

CREATE TABLE Activity (
activity_id INT AUTO_INCREMENT PRIMARY KEY, user_id INT
NOT NULL,
activity_type VARCHAR(50),
activity_date DATETIME DEFAULT CURRENT_TIMESTAMP,
description VARCHAR(255),
FOREIGN KEY (user_id) REFERENCES User(user_id) ON DELETE
CASCADE
);

```

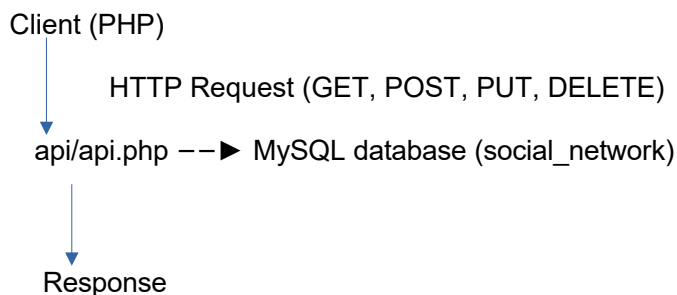
4.Web Application

The technologies used for this application were: PHP, Visual Studio, CSS, MySQL, XAMPP.

The structure of this project consists of an index page, which is the login page, and on that same page there is a registration section where you can sign up if you don't already have an account.

Once logged in, you'll have several sections where you can post (here you can delete your comment or like comments), update your profile, and view the description of what the application includes.

API CONNECTION:



5.Mobile App

The technologies used in this application are: Java, Android Studio, XAMPP, Visual Studio

The structure of this project consists of a main login page, and on that same page, there is a

registration section where you can sign up if you don't already have an account. Once logged in, you'll have access to several sections where you can post (here you can like comments and comment on others' posts), update your profile or delete it, and view the app's description.

6.Desktop Application.

The technologies used for this application were: C#, PHP (for the API), XAMPP, Visual Studio 2022

The structure of this project consists of a main login page, and on that same page, there is a registration section where you can sign up if you don't already have an account.

Once logged in, you'll have access to several sections where you can post (here you can like comments and comment on others' posts), update or delete your profile, and view the app's description.

7.1Passwords

Passwords are never stored in plain text. Both the PHP client and the API use PHP's `password_hash()` and `password_verify()` functions, which apply bcrypt by default with an automatic random salt.

```
// Upon registration
$hash = password_hash($password, PASSWORD_DEFAULT);

// When logging in
if (password_verify($password, $user_data['password_hash'])) { ... }
```

The Java API also hashes with SHA-256 before saving and compares the hash during validation. The database field is named `password_hash` in all cases, not `password`, to make it clear that it is not stored in plain text.

7.2Protection against SQL injection

All database queries use prepared statements with bound parameters. No user-supplied values are concatenated directly into the SQL.

Example in the PHP login script:

```
$stmt = mysqli_prepare($db, "SELECT * FROM User WHERE username=? OR email=? LIMIT 1");
```

```
mysqli_stmt_bind_param($stmt, "ss", $user, $user);  
mysqli_stmt_execute($stmt);
```

Example in the PHP API when creating a user:

```
$stmt = mysqli_prepare($db, "INSERT INTO User (username, email, password_hash,  
account_status) VALUES (?, ?, ?, ?)");  
mysqli_stmt_bind_param($stmt, "ssss", $username, $email, $hash, $status);
```

The Java API uses JDBC's PreparedStatement in the same way; never directly concatenate variables into queries.

7.3 Protection against XSS

All values displayed in HTML are passed through htmlspecialchars() before being rendered. This converts special characters such as < > & ' " into their equivalent HTML entities, preventing a user from injecting JavaScript code.

```
<?= htmlspecialchars($user["username"]) ?>  
<?= htmlspecialchars($error_login) ?>  
<?= nl2br(htmlspecialchars($p["description"])) ?>
```

Using nl2br() alongside htmlspecialchars() is especially important in post content, where the user can include line breaks.

7.4 PHP Session Management

The session is managed using native PHP functions. Upon login, the ID, username, and email are stored in \$_SESSION. Each protected page includes the check_session.php file at the beginning, which verifies that the session exists and regenerates the session ID to prevent session fixation attacks:

```
if (session_status() === PHP_SESSION_NONE) session_start(); if (!  
isset($_SESSION['id'])) {  
    header('Location: ../index.php'); exit();  
}  
session_regenerate_id(true);
```

When logging out, `cerrar\_sesion.php` clears `\$\_SESSION`, deletes the session cookie from the browser, and terminates the session on the server:

```
$_SESSION = array();  
session_destroy();
```

7.5 Session on Android

The Android app stores the user's ID, username, and email in SharedPreferences with `MODE_PRIVATE`, which on Android prevents other apps from accessing this data. When logging out, `editor.clear()` is called, which deletes all stored data.

The `SessionManager` class centralizes all session operations so that there is no direct access to SharedPreferences from fragments and activities.

7.6 Session in C#

In the desktop application, the session is stored in a static variable (`FormLogin.idUsuarioLogado`) that exists in memory only while the application is open. It is automatically destroyed when the main window is closed. No session data is persisted to disk or to the registry.

The "Remember username and password" checkboxes save the data to the system's SharedPreferences. This is convenient for the user, but keep in mind that the password is stored unencrypted.

7.8 CORS headers in the API

The PHP API includes the CORS headers required to allow requests from other origins. In development, they are open to all origins (*) to facilitate testing:

```
header("Access-Control-Allow-Origin: *");  
header("Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS");  
header("Access-Control-Allow-Headers: Content-Type, Authorization");
```

In production, the allowed origin should be restricted to the application's specific domains.

7.9Input Validation

Before processing any data, it is validated on both the client and the server:

- PHP: `filter_var()` with `FILTER_VALIDATE_EMAIL` to validate emails, `trim()` to remove spaces, `strtolower()` to normalize emails
- PHP API: the `campos()` function checks that required fields are not empty before executing any query
- Android: validations in `RegisterActivity` and `LoginActivity` using `TextUtils.isEmpty()` before calling the API
- C#: The `ValidateTextbox` class centralizes validation for empty fields, minimum length, and email format

7.10HTTP Traffic on Android

The Android app uses HTTP instead of HTTPS because the development server does not have an SSL certificate. To enable this, the `AndroidManifest` includes `android:usesCleartextTraffic='true'`. In a production environment, HTTPS should be used and this setting removed.

7.11User and Permission Management

There is no role-based system in place. Any registered user can view all posts and profiles. The only restrictions are:

- Only the user themselves can edit their profile and personal data
- Only the author of a post can delete it (verified on the client, not via the API)
- Only the author of a comment can delete it
- To access any page on the PHP client, you must be logged in

The REST API does not implement token-based authentication, so anyone who knows the URL can make requests directly. This is acceptable in a local development project but would be a problem in production.

8.Deployment

8.2Prerequisites

Component	Version	What it is used for
Java JDK	11 or higher	Compile and run the Java backend and the Android app
Apache Maven	3.6+	Compile the Java project and generate the WAR
Apache Tomcat	10.x	Server where the Java API WAR is deployed
XAMPP / WAMP	Any recent version	Apache + PHP + MySQL server for the web client
MySQL	5.7+ / 8.x	Social_network database
Android Studio	Hedgehog or later	Compile and run the Android app
.NET Framework	4.7.2	Run the C# application on Windows

8.1Deploying the Java backend (API)

Step 1: Compile the WAR

From the Java project folder (where pom.xml is located): mvn
clean package

This generates the red-social.war file in the target/ folder.

Step 2: Deploy to Tomcat

Copy the WAR file to the Tomcat webapps folder:

```
cp target/red-social.war C:\path\tomcat\webapps\
```

Start Tomcat. If it was already running, it will deploy automatically upon detecting the new file. To start it manually from Tomcat's bin folder:

```
startup.bat (Windows)  
./startup.sh (Linux / Mac)
```

Step 3: Verify

Open in your browser:

`http://localhost:8080/red-social/api/usuarios`

It should return a JSON file containing the list of users. If you get a 404 error, check that the WAR file has been deployed correctly to the webapps folder.

8.2 Deploying the PHP Web Client

Step 1: Copy the files

Copy the entire RedSocial folder into the XAMPP htdocs directory (or the WAMP www directory): C:\xampp\htdocs\RedSocial\

Step 2: Create the database

Open phpMyAdmin (`http://localhost/phpmyadmin`) and run the project's SQL script to create the red_social database with all the tables.

Step 3: Start Apache and MySQL

Open the XAMPP control panel and start the Apache and MySQL modules.

Step 4: Access

Open the following URL in your

browser:

`http://localhost/RedSocial/`

8.3 Launch the Android app

emulator

Open the RedSocialApp_fixed project in Android Studio. Create an emulator with API level 33 or higher in the Device Manager. Click Run. The API URL now points to 10.0.2.2:8080, which is the PC's localhost as seen from the emulator.

Physical device

Change the URL in ApiClient.java to the PC's IP address on the local network:

```
private static final String BASE_URL = "http://192.168.1.XXX:8080/red-social/api/";
```

Connect the phone via USB with USB debugging enabled and click Run in Android Studio.

8.4 Deploying the C# application

Open the RedSocial.sln file in Visual Studio. Compile with Ctrl+Shift+B. Run with F5. No additional sever is required. It only requires the Java backend to be running on localhost:8080.

To distribute it without Visual Studio, copy the contents of the bin/Release/ folder to any Windows PC that has .NET Framework 4.7.2 installed.

The presentacion.mp4 file must be in the same folder as the .exe executable for the video button to work.

8.5 Environments

Environment	PHP	Java API	Android
Local development	http://localhost/ SocialNetwork/	http:// localhost:8080/soci al-network/api/	http:// 10.0.2.2:8080/red- social/api/
Physical device	http://192.168.1.X/ SocialNetwork/	http:// 192.168.1.X:8080/ social-network/api/	http:// 192.168.1.X:8080/ social-network/api/
Production	https:// yourdomain.com/	https:// api.yourdomain.com/	https:// api.yourdomain.com/

To switch environments on Android, simply modify the BASE_URL constant in ApiClient.java. In the C# client, you must modify the BaseAddress in each form that makes requests.

10. User and Maintenance Guide

10.1 Step-by-Step Local Installation

Recommended Startup Order

- 1. Start MySQL (from XAMPP or as a service)
- 2. Start Tomcat (from VS Code using the plugin or from startup.bat)
- 3. Verify that the API is responding: <http://localhost:8080/red-social/api/usuarios>
- 4. Start Apache if you are using the PHP client
- 5. Open the Android app in the emulator or run the C# .exe

Test credentials

The database does not come with any pre-existing users. You must register a new account from any of the three clients before you can log in.

10.2 Common Issues

Problem	Probable cause	Solution
No connection to the server (Android)	Tomcat is not running or the URL is incorrect	Start Tomcat and verify that http://localhost:8080/red-social/api/usuarios responds in the PC browser
404 in the Java API	The WAR file is not deployed in webapps	Copy red-social.war to the Tomcat webapps folder and restart
Java compilation error (invalid target release: 17)	The installed Java version is lower than 17	Change the Java version to 11 in pom.xml: <maven.compiler.source>11< /maven.compiler.source>
The Android app won't start (NullPointerException on startup)	The emulator has an API that is too old	Create a new emulator with API level 33 or higher in Device Manager
Error connecting to the database	MySQL is not running or the database name is incorrect	Start MySQL and verify that the database is named red_social in db_connection.php and Conexion.java
Problem	Probable cause	Solution

The profile photo is not being saved in PHP	The uploads/ folder does not exist or does not have write permissions	Create the uploads/ folder inside RedSocial/ and grant it write permissions
The video does not open in C#	The file presentacion.mp4 is not in the executable's folder	Copy presentacion.mp4 to the same folder where the .exe

10.3 Add features

Add a new endpoint to the PHP API

- Open api/api.php and add a new case to the main switch
- Implement the corresponding function in the same file
- Test with a browser or Postman before connecting it to clients

Add a new endpoint to the Java API

- Create a new method in the corresponding Resource class (or create a new class with @Path)
- Compile with `mvn clean package` and redeploy the WAR

Add a new screen in Android

- Create a Java class that extends AppCompatActivity or Fragment
- Create the corresponding XML layout in res/layout/
- Register the Activity in AndroidManifest.xml if it is an Activity
- Add the method to ApiService if you need to call the API
- Add the necessary response models to ApiModels.java

Add functionality in C#

- Add the necessary controls in the Designer for the corresponding form
- Implement the HTTP method in the form using HttpClient
- If a new form is needed, create it as a Form and open it with ShowDialog()

10.2 Database maintenance

If new tables or columns are added, you must update:

- The queries in api/api.php (PHP) or in the Java Resources
- The models in ApiModels.java (Android)
- The model classes in the C# forms that use them

It is recommended to back up the database before making any changes using: `mysqldump -u root red_social > backup.sql`

11. Planned Improvements

The planned improvements for this project include connecting the API and improving the design in the desktop application; making the mobile app more functional, intuitive, and user-friendly; and making the web application more comprehensive with additional sections to provide more options for using or modifying the profile, as well as a more complete API and a more comprehensive database overall.

And I would like to optimize the fluidity of the mobile app.